

Algoritmos Genéticos em Problemas Combinatórios

Nas aulas anteriores, trabalhamos com problemas de **otimização contínua**, nos quais as soluções eram representadas por números reais.

Nesta aula, o cenário muda. Vamos trabalhar com **otimização combinatória**, em que uma solução pode ser uma sequência, uma combinação ou uma permutação de elementos.

Nosso exemplo será o **roteamento de nós de uma rede**, buscando encontrar uma rota de menor custo.

1. Da otimização contínua para a combinatória

Em um problema contínuo, podemos trabalhar com valores como:

1,0 $\rightarrow$ 1,1 $\rightarrow$ 1,2 $\rightarrow$ 1,3

Existe uma infinidade de valores possíveis entre eles.

Em um problema combinatório, trabalhamos com elementos discretos.

Imagine dez nós:

0 1 2 3 4 5 6 7 8 9

Uma solução pode ser:

[0, 7, 2, 8, 5, 1, 9, 4, 3, 6]

Essa sequência representa a rota:

0 $\rightarrow$ 7 $\rightarrow$ 2 $\rightarrow$ 8 $\rightarrow$ 5 $\rightarrow$ 1 $\rightarrow$ 9 $\rightarrow$ 4 $\rightarrow$ 3 $\rightarrow$ 6 $\rightarrow$ 0

O desafio é encontrar **qual sequência produz o menor custo**.

2. Por que o problema é difícil?

O número de possibilidades cresce muito rapidamente.

Para organizar N elementos em uma permutação, temos: N! possibilidades.

Por exemplo:

Nós	Possibilidades
4	24
5	120
10	3.628.800
20	aproximadamente $2,43 \times 10^{18}$

Portanto, testar todas as possibilidades pode se tornar impraticável.

É nesse cenário que o Algoritmo Genético pode ser utilizado: Em vez de testar todas as soluções, procuramos evoluir uma população de soluções boas.

No nosso problema, cada indivíduo representa uma rota.

Por exemplo:

[0, 3, 7, 1, 9, 2, 5, 8, 4, 6]

Uma rota válida precisa:

- conter todos os nós;
- não repetir nenhum nó;
- não deixar nenhum nó de fora.

Válida

[0, 2, 4, 1, 3]

Inválida

[0, 2, 2, 1, 3]

O nó 2 aparece duas vezes e outro elemento foi perdido.

Essa característica determina quais operadores genéticos podemos utilizar.

3. Por que o crossover tradicional pode falhar?

Imagine dois pais:

Pai 1 → [0, 1 | 2, 3]

Pai 2 → [3, 2 | 1, 0]

Um crossover simples poderia produzir:

Filho → [0, 1, 1, 0]

O resultado é inválido:

0 → repetido

1 → repetido

2 → ausente

3 → ausente

Por isso, em problemas de permutação precisamos utilizar operadores específicos.

4. Crossover OX — Ordered Crossover

O **Ordered Crossover (OX)** foi desenvolvido para trabalhar com permutações.

Considere:

Pai 1 → [0 1 | 2 3 4 | 5 6]

Pai 2 → [6 4 | 5 1 0 | 3 2]

Passo 1 — Copiar uma parte do Pai 1

Filho → [_ _ | 2 3 4 | _ _]

Passo 2 — Observar o Pai 2

Ignoramos os elementos que já estão no filho.

Restam:

6, 5, 1, 0

Passo 3 — Completar o filho

Filho → [1 0 | 2 3 4 | 6 5]

O resultado é novamente uma **permutação válida**.

A ideia principal do OX Preservar parte da sequência de um pai e completar o restante respeitando a ordem do outro pai.

5. Mutação Swap

A mutação também precisa preservar a validade da rota.

Na **Mutação Swap**, escolhemos duas posições e simplesmente trocamos seus valores.

Antes:

[0, 1, 2, 3, 4, 5]

Troca entre 1 e 4:

[0, 4, 2, 3, 1, 5]

Todos os elementos continuam presentes. Por isso, a solução continua válida.

Importante

Não podemos simplesmente adicionar um valor aleatório:

$1 + 0,5 = 1,5$

porque o elemento 1,5 não pertence ao conjunto de nós.

6. Avaliando uma solução: o Fitness

Depois de criar uma rota, precisamos descobrir se ela é boa.

Para isso calculamos seu **custo**.

Por exemplo:

$0 \rightarrow 7 \rightarrow 2 \rightarrow 8 \rightarrow 5 \rightarrow \dots \rightarrow 0$

O algoritmo soma o custo de cada enlace:

Custo total =

custo($0 \rightarrow 7$)

+ custo($7 \rightarrow 2$)

+ custo($2 \rightarrow 8$)

+ ...

+ custo($6 \rightarrow 0$)

Quanto menor o custo, **melhor a solução**.

7. Fitness com penalização

Em situações reais, uma rota precisa obedecer a regras.

Por exemplo: Nenhum enlace pode ultrapassar 50 ms.

Imagine que uma rota tenha custo: Custo = 250

mas utilize um enlace de: 65 ms

A rota viola o limite.

Podemos aplicar uma penalização:

Fitness = Custo + Penalização

Se a violação for de:

$65 - 50 = 15$

e a penalização for 1.000:

Penalização = 15×1.000

= 15.000

Então:

Fitness = $250 + 15.000$

= 15.250

A solução deixa de ser atrativa para o algoritmo.